Week 3: Introduction to Classification

- This week focuses on **classification**, a type of machine learning problem where the output variable, y, belongs to a small, finite set of possible values or categories.
- This is distinct from **linear regression**, which predicts a continuous numerical value.

Binary Classification

- **Binary Classification** is a specific type of classification problem where there are only two possible output classes.
- **Examples:**
  - Spam detection (Spam / Not Spam)
  - Financial fraud detection (Fraudulent / Not Fraudulent)
  - Medical diagnosis (Malignant / Benign tumor)
- **Class Representation Conventions:**
  - The two classes are often represented as No/Yes, False/True, or most commonly, **0/1**.
  - **Negative Class:** The "absence" class, conventionally represented by **0** (e.g., not spam, benign tumor).
  - **Positive Class:** The "presence" class, conventionally represented by **1** (e.g., spam, malignant tumor).
  - *Note:* The terms 'positive' and 'negative' are just labels and do not inherently imply 'good' or 'bad'.

Why Linear Regression Fails for Classification

- A naive approach to classification could be to apply linear regression and use a threshold to determine the class.
  - For example, fit a straight line to the data.
  - Set a threshold (e.g., at 0.5 on the predicted y-axis).
  - If the model's output $f(x) \geq 0.5$, predict class 1.
  - If the model's output $f(x) < 0.5$, predict class 0.
- **The Problem with Outliers:**
  1. Linear regression tries to find the best-fitting line for *all* data points.
  2. If an outlier data point is added (e.g., a very large tumor that is clearly malignant), it will pull the best-fit line towards it.

3. This shift in the line can change the **decision boundary** (the point where the prediction switches from 0 to 1).
4. The result is a less accurate model, as the classification threshold is now in a worse position for the original data points. Linear regression is too sensitive to such outliers in a classification context.

Introduction to Logistic Regression

- **Logistic Regression** is a classification algorithm that addresses the shortcomings of using linear regression for this task.
- Despite its name, it is used for **classification**, not regression. The name is due to historical reasons.
- Its key advantage is that its output is always constrained between **0 and 1**, which is ideal for representing probabilities in classification.

The Logistic Regression Model

- **Logistic Regression** is one of the most widely used classification algorithms.
- Unlike linear regression, which fits a straight line, logistic regression fits an **S-shaped curve** to the data. This curve is more suitable for classification tasks where the output is either 0 or 1.

The Sigmoid (Logistic) Function

- The core of the logistic regression model is the **Sigmoid function**, denoted as $g(z)$.
- **Formula:**
  $g(z) = \frac{1}{1+e^{-z}}$
  where e is the mathematical constant ≈ 2.718.
- **Properties of the Sigmoid Function:**
  - It takes any real number z as input and outputs a value between **0 and 1**.
  - If z is a large positive number, e^{-z} approaches 0, so $g(z)$ approaches 1.
  - If z is a large negative number, e^{-z} becomes very large, so $g(z)$ approaches 0.
  - When $z = 0$, $g(0) = \frac{1}{1+e^{0}} = \frac{1}{1+1} = 0.5$.

Constructing the Model

The logistic regression model is built in two steps:

1. **Linear Combination:** First, calculate z using the same linear function as in regression:
   $z = \vec{w} \cdot \vec{x} + b$
2. **Apply Sigmoid Function:** Pass the result z through the sigmoid function to get the final prediction:
   $f_{\vec{w},b}(\vec{x}) = g(z) = g(\vec{w} \cdot \vec{x} + b)$

- **Full Model Equation:**
  $f_{\vec{w},b}(\vec{x}) = \frac{1}{1+e^{-(\vec{w} \cdot \vec{x} + b)}}$

Interpreting the Output 🧐

- The output of the logistic regression model, $f_{\vec{w},b}(\vec{x})$, is interpreted as the **probability** that the output label y is equal to 1, given the input features $\vec{x}$.
- **Formal Notation:**
  $f_{\vec{w},b}(\vec{x}) = P(y = 1 | \vec{x}; \vec{w}, b)$
- **Example:**

- If the model outputs `0.7` for a given tumor size, it means there is a **70% probability** that the tumor is malignant ($y = 1$).
- Since the probabilities must sum to 1, the probability of the tumor being benign ($y = 0$) is:
  $P(y = 0) = 1 - P(y = 1) = 1 - 0.7 = 0.3 \quad (30\% \text{ probability})$

- The next step in using the model is to define a **decision boundary**, which determines how to map these probability outputs (e.g., 0.7) to a final, discrete class prediction (0 or 1).

The Decision Boundary 🗺️

The decision boundary is a key concept in understanding how a logistic regression model makes its final predictions. It is the line or curve that separates the different predicted classes.

From Probability to Prediction

To convert the model's probability output into a discrete prediction (0 or 1), a **threshold** is used.

- The model's output is the probability that $y = 1$:
  $f_{\vec{w},b}(\vec{x}) = P(y = 1|\vec{x})$
- A common threshold is **0.5**.
- **Prediction Rule:**
  - If $f_{\vec{w},b}(\vec{x}) \geq 0.5$, predict $\hat{y} = 1$.
  - If $f_{\vec{w},b}(\vec{x}) < 0.5$, predict $\hat{y} = 0$.

By analyzing the Sigmoid function, $g(z)$, we know that $g(z) \geq 0.5$ only when its input, z, is greater than or equal to 0.

- This simplifies the prediction rule significantly:
  - Predict $\hat{y} = 1$ when $z = \vec{w} \cdot \vec{x} + b \geq 0$.
  - Predict $\hat{y} = 0$ when $z = \vec{w} \cdot \vec{x} + b < 0$.

The **decision boundary** is the line defined by the equation where the model is exactly on the fence:
$\vec{w} \cdot \vec{x} + b = 0$

Types of Decision Boundaries

Linear Decision Boundary

- When using simple features (e.g., x_1, x_2), the decision boundary is a straight line.
- **Example:**
  - Model with two features: $z = w_1x_1 + w_2x_2 + b$.
  - If parameters are $w_1 = 1, w_2 = 1, b = -3$.
  - The decision boundary is the line where $x_1 + x_2 - 3 = 0$, or $x_1 + x_2 = 3$.

Non-Linear Decision Boundary

- By using **polynomial features**, logistic regression can learn complex, non-linear decision boundaries.
- **Example:**
  - Model with polynomial features: $z = w_1 x_1^2 + w_2 x_2^2 + b$.
  - If parameters are $w_1 = 1, w_2 = 1, b = -1$.
  - The decision boundary is the curve where $x_1^2 + x_2^2 - 1 = 0$, or $x_1^2 + x_2^2 = 1$. This is a circle.
- Higher-order polynomial terms can create even more complex shapes, like ellipses or other intricate curves, allowing the model to fit very complex datasets.